

Fuse Machines
AI Fellowship – 2026

Assignment 2

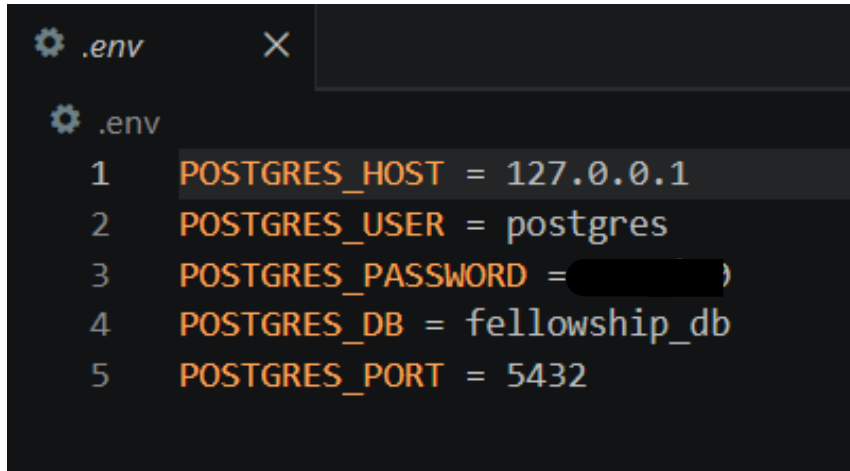
Software Development Concepts

Submitted By:

Sulav Koirala

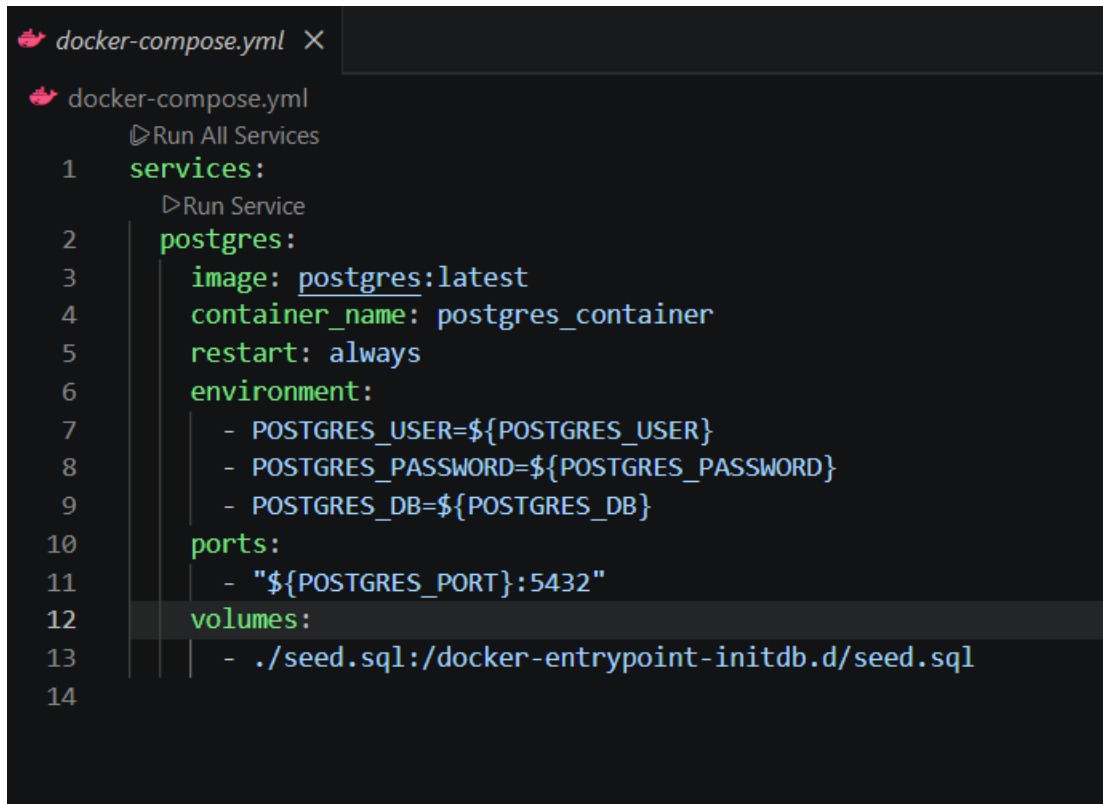
Task 1

.env

A screenshot of a code editor showing a file named .env. The file contains five lines of environment variables for PostgreSQL. The first line is POSTGRES_HOST = 127.0.0.1, the second is POSTGRES_USER = postgres, the third is POSTGRES_PASSWORD = (redacted), the fourth is POSTGRES_DB = fellowship_db, and the fifth is POSTGRES_PORT = 5432. The editor has a dark theme and a sidebar on the left with a gear icon and the filename .env.

```
1 POSTGRES_HOST = 127.0.0.1
2 POSTGRES_USER = postgres
3 POSTGRES_PASSWORD = 
4 POSTGRES_DB = fellowship_db
5 POSTGRES_PORT = 5432
```

.yml

A screenshot of a code editor showing a file named docker-compose.yml. The file defines a service named postgres. The service configuration includes: image: postgres:latest, container_name: postgres_container, restart: always, environment variables for POSTGRES_USER, POSTGRES_PASSWORD, and POSTGRES_DB using \${}, ports: ["\${POSTGRES_PORT}:5432"], and volumes: [./seed.sql:/docker-entrypoint-initdb.d/seed.sql]. The editor has a dark theme and a sidebar on the left with a Docker icon and the filename docker-compose.yml.

```
1 services:
2   postgres:
3     image: postgres:latest
4     container_name: postgres_container
5     restart: always
6     environment:
7       - POSTGRES_USER=${POSTGRES_USER}
8       - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
9       - POSTGRES_DB=${POSTGRES_DB}
10    ports:
11      - "${POSTGRES_PORT}:5432"
12    volumes:
13      - ./seed.sql:/docker-entrypoint-initdb.d/seed.sql
14
```

Short Reflection:

1. Why is .env better than writing passwords in code?

Ans: Hardcoding passwords directly into code is honestly just asking for trouble. If I push my code to GitHub, anyone can see my database password and other details like hostname, username, port etc. or API key sitting right there. Using an .env file means sensitive stuff stays out of the codebase entirely. It also makes switching between environments way easier, I don't have to dig through files changing values manually every time.

2. Why is treating the database as a separate service useful?

Ans: Treating the database like a separate, swappable service rather than something put into the app just makes everything more flexible. If my local PostgreSQL breaks or I need to switch to MySQL in production, the app shouldn't care, it just reads a URL and connects. It also made me realize how tightly coupled apps can get when developers assume the database is always “just there”.

3. Why does Docker make development and production similar?

Ans: Before Docker, I'd write code that worked on my laptop and then it'd randomly break on the server. Docker packages everything, the OS libraries, dependencies, runtime all into one container. So development and production are basically running the same environment. That working on machine only problem becomes way less of a thing.

Terminal:

```
(venv) PS C:\Users\Lenovo\OneDrive\Desktop\FuseMachines AI Fellowship\assignments\Week 2> docker-compose up -d  
[+] up 1/1  
✓ Container postgres_container Running
```

```
(venv) PS C:\Users\Lenovo\OneDrive\Desktop\FuseMachines AI Fellowship\assignments\Week 2> docker exec -it postgres_container psql -U postgres -d fellowship_db  
psql (18.1 (Debian 18.1-1.pgdg13+2))  
Type "help" for help.
```

```
fellowship_db=# \dt  
List of tables  
Schema | Name | Type | Owner  
-----+-----+-----+-----  
public | customers | table | postgres  
public | employees | table | postgres  
public | offices | table | postgres  
public | orderdetails | table | postgres  
public | orders | table | postgres  
public | payments | table | postgres  
public | productlines | table | postgres  
public | products | table | postgres  
(8 rows)
```

```
fellowship_db=# select count(*) from products;  
count  
-----  
110  
(1 row)
```

Task 2 and 3

```
main.py ×  
main.py > ...  
1  from fastapi import FastAPI  
2  import router,task3_router  
3  
4  app = FastAPI()  
5  app.include_router(router.router)  
6  app.include_router(task3_router.router)
```

```
logger.py ×  
logger.py > ...  
1  import logging  
2  import sys  
3  
4  logging.basicConfig(level=logging.INFO,  
5                      format="%asctime)s - %(levelname)s - %(message)s",  
6                      handlers=[  
7                          logging.FileHandler("app.log"),  
8                          logging.StreamHandler(sys.stdout)  
9                      ])  
10 logger = logging.getLogger("CustomerAPI")
```

```
config.py ×  
config.py > Settings  
1  from pydantic_settings import BaseSettings  
2  
3  class Settings(BaseSettings):  
4      POSTGRES_HOST : str  
5      POSTGRES_USER : str  
6      POSTGRES_PASSWORD : str  
7      POSTGRES_DB : str  
8      POSTGRES_PORT : str  
9  
10     class Config:  
11         env_file=".env"  
12  
13     settings=Settings()
```

```

schemas.py
schemas.py > ...
1  from pydantic import BaseModel
2  from typing import Optional, List
3
4  class OrderSchema(BaseModel):
5      order_number: int
6      status: str
7      class Config:
8          from_attributes = True
9  class PaymentSchema(BaseModel):
10     check_number: str
11     amount: float
12     class Config:
13         from_attributes = True
14
15     class CustomerBase(BaseModel):
16         customer_name : str
17         contact_last_name : str
18         contact_first_name : str
19         phone : str
20         address_line1: str
21         city : str
22         country : str
23         class Config:
24             from_attributes = True
25
26     class CustomerCreate(CustomerBase):
27         customer_number : int
28         class Config:
29             from_attributes = True
30
31     class CustomerUpdate(CustomerBase):
32         __annotations__ = {k: Optional[v] for k, v in CustomerBase.__annotations__.items()}
33
34     class CustomerOut(CustomerBase):
35         customer_number : int
36         orders : List[OrderSchema] = []
37         payments : List[PaymentSchema] = []
38         class Config:

```

```

database.py
database.py > ...
1  from sqlalchemy import create_engine
2  from sqlalchemy.orm import sessionmaker
3  from sqlalchemy.ext.declarative import declarative_base
4  from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
5  from config import settings
6  from logger import logger
7
8  Base=declarative_base()
9
10 #For Task 2
11 sqlalchemy_db_url=f"postgresql://{settings.POSTGRES_USER}:{settings.POSTGRES_PASSWORD}@{settings.POSTGRES_HOST}:{settings.POSTGRES_PORT}/{settings.POSTGRES_DB}"
12 try:
13     engine=create_engine(sqlalchemy_db_url)
14     session_local=sessionmaker(autocommit=False, autoflush=False, bind=engine)
15     logger.info("Database connection established succesfully. ")
16 except Exception as e:
17     logger.error(f'Database connection failed: {str(e)}')
18
19 def get_db():
20     db=session_local()
21     try:
22         yield db
23     finally:
24         db.close()
25

```

```

#For Task 3
async_url = f"postgresql+asyncpg://{settings.POSTGRES_USER}:{settings.POSTGRES_PASSWORD}@{settings.POSTGRES_HOST}:{settings.POSTGRES_PORT}/{settings.POSTGRES_DB}"
try:
    async_engine = create_async_engine(async_url)
    AsyncSessionLocal = async_sessionmaker(bind=async_engine, class_=AsyncSession)
    logger.info("Database connection established succesfully. ")
except Exception as e:
    logger.error(f'Database connection failed: {str(e)}')

async def get_async_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
        finally:
            await session.close()

def get_async_session_factory():
    return AsyncSessionLocal

```

```

models.py X
models.py > ...
1  from sqlalchemy import Column, Integer, String, Numeric, ForeignKey, Date, Text
2  from sqlalchemy.orm import relationship
3  from database import Base
4
5  class Customer(Base):
6      __tablename__ = "customers"
7
8      customer_number = Column("customerNumber", Integer, primary_key=True, nullable=False)
9      customer_name = Column("customerName", String(50), nullable=False)
10     contact_last_name = Column("contactLastName", String(50), nullable=False)
11     contact_first_name = Column("contactFirstName", String(50), nullable=False)
12     phone = Column(String(50), nullable=False)
13     address_line1 = Column("addressLine1", String(50), nullable=False)
14     address_line2 = Column("addressLine2", String(50), nullable=True)
15     city = Column(String(50), nullable=False)
16     state = Column(String(50), nullable=True)
17     postal_code = Column("postalCode", String(15), nullable=True)
18     country = Column(String(50), nullable=False)
19     sales_rep_employee_number = Column("salesRepEmployeeNumber", Integer, ForeignKey("employees.employeeNumber"), nullable=True)
20     credit_limit = Column("creditLimit", Numeric(10, 2), nullable=True)
21
22     orders = relationship("Order", back_populates="customer")
23     payments = relationship("Payment", back_populates="customer")
24     employee = relationship("Employee", back_populates="customer")
25
26
27     class Order(Base):
28         __tablename__ = "orders"
29
30         order_number = Column("orderNumber", Integer, primary_key=True, nullable=False)
31         order_date = Column("orderDate", Date, nullable=False)
32         required_date = Column("requiredDate", Date, nullable=False)
33         shipped_date = Column("shippedDate", Date, nullable=True)
34         status = Column(String(15), nullable=False)
35         comments = Column(Text, nullable=True)
36
37         customer_number = Column("customerNumber", Integer, ForeignKey("customers.customerNumber"), nullable=False)
38         customer = relationship("Customer", back_populates="orders")
39
40     class Payment(Base):
41         __tablename__ = "payments"
42
43         customer_number = Column("customerNumber", Integer, ForeignKey("customers.customerNumber"), primary_key=True, nullable=False)
44         check_number = Column("checkNumber", String(50), primary_key=True, nullable=False)
45         payment_date = Column("paymentDate", Date, nullable=False)
46         amount = Column(Numeric(10, 2), nullable=False)
47
48         customer = relationship("Customer", back_populates="payments")
49
50 #Only defined the model with table name and primary key because i don't need other attributes to find count in Task 3
51 class Product(Base):
52     __tablename__ = "products"
53     product_code = Column("productCode", String(15), primary_key=True)
54
55 class Employee(Base):
56     __tablename__ = "employees"
57     employee_number = Column("employeeNumber", Integer, primary_key=True)
58     customer = relationship("Customer", back_populates="employee")
59
60 class Office(Base):
61     __tablename__ = "offices"
62     office_code = Column("officeCode", String(10), primary_key=True)
63
64 class OrderDetail(Base):
65     __tablename__ = "orderdetails"
66     order_number = Column("orderNumber", Integer, primary_key=True)
67
68 class Productline(Base):
69     __tablename__ = "productlines"
70     product_line = Column("productLine", String(50), primary_key=True)

```

```
app.log X
app.log
1 2026-05-08 12:15:58,027 - ERROR - Database connection failed: No module named 'psycopg2'
2 2026-05-08 12:17:36,144 - ERROR - Database connection failed: No module named 'psycopg2'
3 2026-05-08 12:18:19,107 - ERROR - Database connection failed: No module named 'psycopg2'
4 2026-05-08 12:18:53,648 - ERROR - Database connection failed: No module named 'psycopg2'
5 2026-05-08 12:19:40,668 - INFO - Database connection established successfully.
6 2026-05-08 12:20:16,686 - INFO - Searching for customer ID: 1
7 2026-05-08 12:24:54,058 - INFO - Database connection established successfully.
8 2026-05-08 12:25:10,445 - INFO - Searching for customer ID: 1
9 2026-05-08 12:29:41,900 - INFO - Database connection established successfully.
10 2026-05-08 12:30:07,236 - INFO - Searching for customer ID: 1
11 2026-05-08 12:32:06,848 - INFO - Database connection established successfully.
12 2026-05-08 12:32:19,084 - INFO - Searching for customer ID: 1
13 2026-05-08 12:32:19,126 - WARNING - Customer 1 not found.
14 2026-05-08 12:32:41,184 - INFO - Searching for customer ID: 103
15 2026-05-08 12:34:34,498 - INFO - Database connection established successfully.
16 2026-05-08 12:34:42,085 - INFO - Database connection established successfully.
17 2026-05-08 12:34:50,165 - INFO - Searching for customer ID: 103
18 2026-05-08 12:36:49,045 - INFO - Database connection established successfully.
19 2026-05-08 12:37:02,365 - INFO - Searching for customer ID: 1
20 2026-05-08 12:37:51,252 - INFO - Database connection established successfully.
21 2026-05-08 12:37:56,937 - INFO - Searching for customer ID: 1
22 2026-05-08 12:37:56,987 - WARNING - Customer 1 not found.
23 2026-05-08 12:38:02,361 - INFO - Searching for customer ID: 103
24 2026-05-08 12:38:08,833 - INFO - Searching for customer ID: 112
25 2026-05-08 12:38:57,304 - INFO - Fetching customers with offset 0 and limit 10
26 2026-05-08 12:39:07,485 - INFO - Fetching customers with offset 0 and limit 5
27 2026-05-08 12:39:14,484 - INFO - Fetching customers with offset 2 and limit 5
28 2026-05-08 12:41:15,115 - INFO - Database connection established successfully.
29 2026-05-08 12:40:46,510 - INFO - Database connection established successfully.
```

```
(venv) PS C:\Users\Lenovo\OneDrive\Desktop\FuseMachines_AI Fellowship\assignments\Week 2> uvicorn main:app --reload
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [12652] using StatReload
2026-05-08 16:00:40,656 - INFO - Database connection established successfully.
2026-05-08 16:00:40,703 - INFO - Database connection established successfully.
INFO: Started server process [22176]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

```
router.py X
router.py > ...
1 #For Task 2
2 from fastapi import APIRouter, Depends, HTTPException, status
3 from sqlalchemy.orm import Session
4 from typing import List
5 import crud, schemas, database
6 from logger import logger
7
8 router = APIRouter(prefix="/customers", tags=['Customers'])
9
10 @router.get("/{customer_id}", response_model=schemas.CustomerOut)
11 def get_customer(customer_id: int, db: Session = Depends(database.get_db)):
12     db_customer = crud.get_customer(db, customer_id=customer_id)
13     if db_customer is None:
14         logger.warning(f"Customer {customer_id} not found.")
15         raise HTTPException(status_code=404, detail="Customer not found")
16     return db_customer
17
18 @router.get("/", response_model=List[schemas.CustomerOut])
19 def get_customers(skip: int = 0, limit: int = 10, db: Session = Depends(database.get_db)):
20     return crud.get_customers(db, skip=skip, limit=limit)
21
22 @router.post("/create", status_code=status.HTTP_201_CREATED)
23 def create_customer(customer: schemas.CustomerCreate, db: Session = Depends(database.get_db)):
24     existing_customer = crud.get_customer(db, customer_id=customer.customer_number)
25     if existing_customer:
26         logger.error(f"Failed to create: Customer ID {customer.customer_number} already exists.")
27         raise HTTPException(
28             status_code=400,
29             detail="Customer number already registered"
30         )
31
32     logger.info(f"Creating new customer: {customer.customer_name}")
33     return crud.create_customer(db=db, customer=customer)
34
```



```

task3_router.py X
task3_router.py > ...
1 from sqlalchemy.ext.asyncio import AsyncSession
2 import asyncio
3 from fastapi import APIRouter, Depends
4 import models, crud, database
5 from logger import logger
6
7 router = APIRouter(prefix="/count", tags=['Counts'])
8
9 @router.get("/customers")
10 async def count_customers(db: AsyncSession = Depends(database.get_async_db)):
11     logger.info("Request received: GET /count/customers")
12     count = await crud.get_count(db, models.Customer)
13     return {"customers": count}
14
15 @router.get("/orders")
16 async def count_orders(db: AsyncSession = Depends(database.get_async_db)):
17     logger.info("Request received: GET /count/orders")
18     count = await crud.get_count(db, models.Order)
19     return {"orders": count}
20
21 @router.get("/payments")
22 async def count_payments(db: AsyncSession = Depends(database.get_async_db)):
23     logger.info("Request received: GET /count/payments")
24     count = await crud.get_count(db, models.Payment)
25     return {"payments": count}
26
27 @router.get("/employees")
28 async def count_employees(db: AsyncSession = Depends(database.get_async_db)):
29     logger.info("Request received: GET /count/employees")
30     count = await crud.get_count(db, models.Employee)
31     return {"employees": count}
32
33 @router.get("/products")
34 async def count_products(db: AsyncSession = Depends(database.get_async_db)):
35     logger.info("Request received: GET /count/products")
36     count = await crud.get_count(db, models.Product)
37     return {"products": count}
38

```

```

task3_router.py > ... get overall counts
@router.get("/productlines")
async def count_productlines(db: AsyncSession = Depends(database.get_async_db)):
    logger.info("Request received: GET /count/productlines")
    count = await crud.get_count(db, models.ProductLine)
    return {"productlines": count}

@router.get("/overall_counts")
async def get_overall_counts(session_factory=Depends(database.get_async_session_factory)):
    logger.info("Request: Starting concurrent counts")

    async def count(model):
        async with session_factory() as session:
            return await crud.get_count(session, model)

    results = await asyncio.gather(
        count(models.Customer),
        count(models.Order),
        count(models.Product),
        count(models.Employee),
        count(models.Office),
        count(models.Payment),
        count(models.OrderDetail),
        count(models.ProductLine),
    )

    return {
        "customers": results[0],
        "orders": results[1],
        "products": results[2],
        "employees": results[3],
        "offices": results[4],
        "payments": results[5],
        "orderdetails": results[6],
        "productlines": results[7],
    }

```

```

crud.py  X
crud.py > ...
1  from sqlalchemy.orm import Session, joinedload
2  from sqlalchemy.ext.asyncio import AsyncSession
3  from sqlalchemy.future import select
4  from sqlalchemy import func
5  import models, schemas
6  from logger import logger
7
8  #task 2
9  def get_customer(db: Session, customer_id: int):
10     logger.info(f"Searching for customer ID: {customer_id}")
11     return db.query(models.Customer).options(joinedload(models.Customer.orders), joinedload(models.Customer.payments))\
12         .filter(models.Customer.customer_number == customer_id).first()
13
14  def get_customers(db: Session, skip: int = 0, limit: int = 10):
15     logger.info(f"Fetching customers with offset {skip} and limit {limit}")
16     return db.query(models.Customer).options(joinedload(models.Customer.orders), joinedload(models.Customer.payments)).offset(skip).limit(limit).all()
17
18  def create_customer(db: Session, customer: schemas.CustomerCreate):
19     db_customer = models.Customer(**customer.model_dump())
20     db.add(db_customer)
21     db.commit()
22     db.refresh(db_customer)
23     logger.info(f"Created new customer with ID: {db_customer.customer_number}")
24     return db_customer
25
26  #task 3
27  async def get_count(db: AsyncSession, model):
28     model_name = model.__name__
29     logger.info(f"Starting count query for {model_name}")
30     try:
31         result = await db.execute(select(func.count()).select_from(model))
32         count = result.scalar() or 0
33         logger.info(f"Completed count for {model_name}: {count}")
34         return count
35     except Exception as e:
36         logger.error(f"Error counting {model_name}: {str(e)}")
37         return 0

```

Customers			^
GET	/customers/{customer_id}	Get Customer	▼
GET	/customers/	Get Customers	▼
POST	/customers/create	Create Customer	▼
Counts			^
GET	/count/customers	Count Customers	▼
GET	/count/orders	Count Orders	▼
GET	/count/payments	Count Payments	▼
GET	/count/employees	Count Employees	▼
GET	/count/products	Count Products	▼
GET	/count/offices	Count Offices	▼
GET	/count/orderdetails	Count Orderdetails	▼
GET	/count/productlines	Count Productlines	▼
GET	/count/overall_counts	Get Overall Counts	▼

200

Response body

```
{
  "customers": 124,
  "orders": 326,
  "products": 110,
  "employees": 23,
  "offices": 7,
  "payments": 273,
  "orderdetails": 2996,
  "productlines": 7
}
```



Download

Response headers

Request URL

http://127.0.0.1:8000/customers/103

Server response

Code Details

200

Response body

```
{
  "customer_name": "Atelier graphique",
  "contact_last_name": "Schmitt",
  "contact_first_name": "Carine ",
  "phone": "40.32.2555",
  "address_line1": "54, rue Royale",
  "city": "Nantes",
  "country": "France",
  "customer_number": 103,
  "orders": [
    {
      "order_number": 10123,
      "status": "Shipped"
    },
    {
      "order_number": 10298,
      "status": "Shipped"
    },
    {
      "order_number": 10345,
      "status": "Shipped"
    }
  ],
  "payments": [
    {
      "check_number": "OM314933",
      "amount": 1676.14
    },
    {
      "check_number": "3W555205"
    }
  ]
}
```



Download

Request URL

http://127.0.0.1:8000/customers/create

Server response

Code Details

400 Error: Bad Request

Undocumented

Response body

```
{
  "detail": "Customer number already registered"
}
```



Download

Request URL

http://127.0.0.1:8000/customers/?skip=0&limit=3

Server response

Code Details

200

Response body

```
{
  {
    "customer_name": "Australian Collectors, Co.",
    "contact_last_name": "Ferguson",
    "contact_first_name": "Peter",
    "phone": "03 9520 4555",
    "address_line1": "636 St Kilda Road",
    "city": "Melbourne",
    "country": "Australia",
    "customer_number": 114,
    "orders": [
      {
        "order_number": 10120,
        "status": "Shipped"
      },
      {
        "order_number": 10125,
        "status": "Shipped"
      },
      {
        "order_number": 10223,
        "status": "Shipped"
      },
      {
        "order_number": 10342,
        "status": "Shipped"
      }
    ]
  },
  {
    "customer_name": "Atelier graphique",
    "contact_last_name": "Schmitt",
    "contact_first_name": "Carine ",
    "phone": "40.32.2555",
    "address_line1": "54, rue Royale",
    "city": "Nantes",
    "country": "France",
    "customer_number": 103,
    "orders": [
      {
        "order_number": 10123,
        "status": "Shipped"
      },
      {
        "order_number": 10298,
        "status": "Shipped"
      },
      {
        "order_number": 10345,
        "status": "Shipped"
      }
    ],
    "payments": [
      {
        "check_number": "OM314933",
        "amount": 1676.14
      },
      {
        "check_number": "3W555205"
      }
    ]
  }
]
```



Download

Response headers

```
content-length: 1634
content-type: application/json
date: Fri, 08 May 2026 10:55:49 GMT
```